CLOUD COMPUTING · AMAZON WEB SERVICES

AWS Basics

Interview & Learning Guide

Core concepts · Most-used services · Comparisons · Interview Q&A

- IAM, VPC, EC2, S3, RDS, DynamoDB, Lambda, ECS/EKS
- Load balancing, autoscaling, CloudWatch, CloudTrail, CloudFront
- SQS, SNS, API Gateway, Step Functions, EventBridge
- Well-Architected Framework, shared responsibility, common architectures

Study notes for interviews and day-to-day cloud work

Contents

1. Cloud & AWS fundamentals
2. Identity, security, and governance
3. Compute (EC2, Lambda, containers)
4. Storage (S3, EBS, EFS, Glacier)
5. Databases (RDS, Aurora, DynamoDB, ElastiCache)
6. Networking & content delivery
7. Application integration & APIs
8. Monitoring, logging, and audit
9. DevOps, IaC, and CI/CD
10. Analytics, messaging, and extra high-frequency services
11. Well-Architected Framework & shared responsibility
12. Service comparison cheat sheets
13. Common architectures you should be able to draw
14. Interview questions and model answers
15. Quick study checklist

How to use this guide

Read sections 1–2 for vocabulary. Memorize the service tables (what / when / interview trap). Practice drawing the architectures in section 13 out loud. Drill section 14 until you can answer in 60–90 seconds. AWS names change; principles (HA, least privilege, decoupling, managed services) do not.

1. Cloud & AWS fundamentals

Amazon Web Services (AWS) is a public cloud platform: on-demand compute, storage, databases, networking, and higher-level services billed mostly as you go. Interviews test whether you can pick the **right service for a requirement** (latency, durability, consistency, cost, ops burden) and design for **failure** (multi-AZ, retries, decoupling).

1.1 Cloud models

Model	You manage	Typical use
IaaS	OS, runtime, app, data (provider: hardware, hypervisor, network fabric)	EC2, VPC — lift-and-shift, custom stacks
PaaS / CaaS	App and data; platform handles servers/orchestration pieces	Elastic Beanstalk, ECS/EKS, App Runner
FaaS / serverless	Code and config; no servers to patch	Lambda, API Gateway, DynamoDB, S3
SaaS	Usually just usage/config of a finished product	Amazon Connect, WorkSpaces, third-party SaaS on AWS

1.2 Deployment models

- **Public cloud:** shared AWS regions (most interview answers assume this).
- **Hybrid:** on-prem + AWS (Direct Connect, Site-to-Site VPN, Outposts, Storage Gateway).
- **Multi-cloud:** AWS plus another provider — talk about portable design, not brand loyalty.
- **Private:** dedicated hardware (Outposts, Local Zones for low latency near users).

1.3 Global infrastructure vocabulary (must-know)

Term	Meaning	Interview hook
Region	Geographic cluster of AZs (e.g. us-east-1). Isolated from other regions.	Pick region for latency, compliance, service availability, cost.
Availability Zone (AZ)	One or more discrete data centers with independent power/networking in a Region.	HA = multi-AZ. Single-AZ is a common failure mode.
Edge location / PoP	CloudFront and Route 53 presence closer to users.	CDN caches at the edge; not the same as a Region.
Local Zone / Wavelength	AWS compute/storage nearer to metro or 5G networks.	Ultra-low latency apps (gaming, media, telco).
Account	Billing and isolation boundary. Use Organizations + multiple accounts.	Prod vs non-prod accounts beat one giant account.

1.4 CAP theorem in one sentence for AWS interviews

Distributed systems trade consistency, availability, and partition tolerance. DynamoDB default is **eventually consistent** reads (cheaper, faster) with optional strongly consistent reads. RDS/Aurora give stronger relational consistency within a primary; replicas are typically eventually consistent. Say this when comparing SQL vs NoSQL.

1.5 Pricing mental model

- **On-Demand:** no commitment; highest unit price; good for spiky/unknown load.
- **Savings Plans / Reserved Instances:** 1–3 year commitment for steady compute.
- **Spot:** spare capacity, can be interrupted — batch, CI, fault-tolerant workers.
- **Data transfer:** inbound usually free; outbound (egress) and cross-AZ/Region often billed.

- **Managed services:** you pay a premium vs self-managed EC2 to reduce ops and risk.

Free-tier and exact prices change. Interviews care that you **name the cost drivers** (hours, GB-month, requests, provisioned RCU/WCU, NAT Gateway hours + GB, CloudWatch logs ingestion).

2. Identity, security, and governance

Almost every AWS interview includes IAM. Default stance: **deny all, least privilege, no long-lived keys on people if SSO exists, never embed credentials in code.**

Service	What it is / primary use	Interview / exam notes
IAM	Users, groups, roles, policies. Authentication + authorization for AWS APIs.	Prefer roles over users. Identity-based vs resource-based policies.
IAM Identity Center (SSO)	Workforce SSO into AWS accounts and apps.	Modern replacement for many IAM users.
AWS Organizations	Multi-account hierarchy, consolidated billing, SCPs.	SCPs are guardrails (max permissions), not grants.
KMS	Create and control encryption keys; integrates with S3, EBS, RDS, etc.	CMK vs AWS-owned keys; key policies + IAM; envelope encryption.
Secrets Manager	Store/rotate secrets (DB passwords, API keys) with IAM and audit.	Vs SSM Parameter Store: rotation + secrets-focused; Parameter Store cheaper for config.
Systems Manager Parameter Store	Hierarchical config and optional SecureString parameters.	Standard vs advanced parameters; often paired with EC2/Lambda.
Cognito	User pools (app users) and identity pools (federate to AWS credentials).	Not the same as IAM users. Mobile/web customer identity.
WAF / Shield / GuardDuty / Inspector / Macie / Security Hub	Edge web ACL; DDoS; intelligent threat findings; vuln scanning; S3 PII; central findings.	Know the one-liner for each; don't mix GuardDuty (threats) with Inspector (vulns).
CloudTrail	Account activity / API audit log (who did what, when, from where).	Enable org trail to S3 + CloudWatch. Distinct from CloudWatch metrics.
Config	Record resource configuration and evaluate rules over time.	Compliance drift detection; pairs with SCPs and Security Hub.
ACM	Provision TLS certificates for ALB, CloudFront, API Gateway.	Public certs are free; private CA is a separate product.

2.1 IAM policy types (say this clearly)

- **Identity-based:** attached to user/group/role — what that principal can do.
- **Resource-based:** on the resource (S3 bucket policy, SQS, KMS, Lambda) — who can access it.
- **Permission boundaries:** max permissions a role/user can ever get (delegation safety).
- **SCPs:** organization-wide ceiling; do not grant access by themselves.
- **Session policies:** further restrict a temporary session (STS AssumeRole).

Evaluation logic (simplified): explicit deny always wins. Then allow must exist. SCPs and boundaries can still block an allow. For S3, bucket policy + IAM both matter.

2.2 Roles vs users vs instance profiles

IAM **users** are long-lived identities (avoid for humans when SSO is available). IAM **roles** are assumed and receive temporary credentials via STS — used by EC2 (instance profile), Lambda, ECS tasks, cross-account access, and federated users. An **instance profile** is the wrapper that lets an EC2 instance assume a role. Cross-account: Account A role trusts Account B principal; B assumes the role.

3. Compute

Service	What it is / primary use	Interview / exam notes
EC2	Virtual machines (instances). Full OS control. AMIs, instance types, key pairs, user data.	Know family (T burstable, M general, C compute, R memory, I storage, G/P GPU).
EBS (with EC2)	Network-attached block volumes for instances. Snapshots to S3.	gp3 default general purpose; io2 for high IOPS; independent lifecycle from instance if not delete-on-terminate.
Auto Scaling (ASG)	Maintain desired EC2 capacity; scale on CPU, ALB requests, custom metrics, schedules.	Launch template + ASG + ALB = classic HA web tier.
Elastic Load Balancing	ALB (L7 HTTP/HTTPS, path/host), NLB (L4 TCP/UDP/TLS, static IP), GWLB (appliances), CLB (legacy).	Prefer ALB for HTTP microservices; NLB for extreme performance / static IP / non-HTTP.
Lambda	Run code without servers. Event-driven. 15-minute max. Pay per request + GB-seconds.	Cold starts, VPC ENI, concurrency limits, idempotency, /tmp ephemeral.
ECS	Run Docker containers. Launch types: Fargate (serverless tasks) or EC2 (you manage instances).	Task definition, service, cluster. IAM task roles.
EKS	Managed Kubernetes control plane. Worker nodes on EC2 or Fargate.	Choose when you need k8s ecosystem; more ops than ECS/Fargate.
Fargate	Serverless compute engine for ECS/EKS — no EC2 node management.	Great interview answer for “containers without servers.”
Elastic Beanstalk	PaaS: upload app, AWS provisions ELB/ASG/EC2/RDS options.	Fast for simple apps; less flexible than raw IaC.
App Runner / Lightsail	App Runner: source-to-service containers/web apps. Lightsail: simple VPS + fixed pricing.	Lightsail is the “easy/cheap VPS”; not for complex enterprise designs.
Batch / EMR / SageMaker (compute-ish)	Batch: scheduled container/EC2 jobs. EMR: Hadoop/Spark. SageMaker: ML training/hosting.	Name them when the prompt is big data or ML, not generic web apps.

3.1 EC2 details interviewers love

- **Placement groups:** cluster (low latency), spread (separate hardware), partition (Hadoop/Cassandra).
- **Hibernate / stop / terminate:** stop keeps EBS, loses instance store; terminate can delete volumes.
- **Instance store:** ephemeral local disk — high IOPS, data dies with instance/stop (type-dependent).
- **ENI / Elastic IP:** Elastic IP is a static public IPv4 (charges if unused). ENI for extra NICs/IPs.
- **User data:** bootstrap script on first boot; cloud-init. Prefer AMI baking + SSM for production.
- **IMDSv2:** session-oriented metadata; mitigate SSRF credential theft.

3.2 Lambda vs EC2 vs containers — pick in 10 seconds

Need	Prefer	Why
Short event-driven work, spiky, little ops	Lambda	No servers; scales to zero; 15-min cap
Long-running process, custom OS, GPU, Windows	EC2	Full control; always-on cheaper at high duty cycle
Microservices with Docker, team already on k8s	EKS	Portability and k8s tooling
Docker without k8s complexity	ECS + Fargate	AWS-native, less cluster ops
HTTP API in front of functions	API Gateway + Lambda	Classic serverless web/API

4. Storage

Service	What it is / primary use	Interview / exam notes
S3	Object storage. Virtually unlimited. 11 nines durability design. HTTP(S) API. Buckets + keys.	Not a filesystem. Strong read-after-write for new PUTs. Storage classes matter.
S3 Glacier / Instant Retrieval / Flexible / Deep Archive	Archive classes (now S3 storage classes, not a totally separate product in most answers).	Retrieval time vs cost. Deep Archive cheapest, hours to restore.
EBS	Block storage for one AZ, attached to one instance (io2 Block Express / multi-attach exceptions).	Snapshots are incremental, stored in S3, restore to new volume.
EFS	Regional NFS (NFSv4) shared filesystem for many Linux instances/containers/Lambda.	Multi-AZ by default. Bursting vs provisioned/elastic throughput.
FSx	Managed file systems: FSx for Windows (SMB), Lustre, NetApp ONTAP, OpenZFS.	Windows file shares → FSx for Windows, not EFS.
Storage Gateway	Hybrid: file/volume/tape gateway between on-prem and S3/Glacier/FSx.	Migration and backup hybrid patterns.
AWS Backup	Central backup policies across EBS, RDS, EFS, DynamoDB, etc.	Better answer than “custom snapshot scripts” for enterprise.
ECR	Private Docker image registry (used with ECS/EKS/Lambda).	Scan images; IAM for pull/push.

4.1 S3 storage classes (high-frequency)

Class	When to use	Watch-outs
S3 Standard	Frequent access, low latency	Highest storage $ among hot classes
Standard-IA / One Zone-IA	Infrequent access, still millisecond GET	Retrieval fee; One Zone not multi-AZ
Glacier Instant Retrieval	Archive that must GET in milliseconds	Higher retrieval $
Glacier Flexible Retrieval	Minutes–hours restore OK	Not for real-time apps
Glacier Deep Archive	Long-term compliance archive	12–48h typical restore
Intelligent-Tiering	Unknown or changing access patterns	Small monitoring fee; great default for mixed

4.2 S3 features to name in interviews

- **Versioning + MFA Delete** for ransomware/accidental overwrite protection.
- **Lifecycle rules** to transition/expire objects (cost control).
- **Replication:** CRR (cross-region) / SRR (same-region) — versioning required.
- **Encryption:** SSE-S3, SSE-KMS, SSE-C, client-side. Bucket keys reduce KMS cost.
- **Block Public Access** account/bucket settings — default secure posture.
- **Presigned URLs** for temporary object access without making the bucket public.
- **Object Lock (WORM)** for compliance.
- **Event notifications** to SQS/SNS/Lambda/EventBridge.
- **Transfer Acceleration / Multipart upload** for large/far clients.

5. Databases

Service	What it is / primary use	Interview / exam notes
RDS	Managed relational: MySQL, PostgreSQL, MariaDB, SQL Server, Oracle. Multi-AZ standby, read replicas.	You don't SSH the OS. Backups, patching windows, parameter groups.
Aurora	AWS cloud-native MySQL/PostgreSQL-compatible. Shared storage, fast replicas, Aurora Serverless v2.	Often the "managed SQL at scale" answer. Global Database for low-RPO DR.
DynamoDB	Managed NoSQL key-value / document. Single-digit ms. On-demand or provisioned capacity.	Partition key design is everything. Streams, TTL, GSI/LSI, DAX, Global Tables.
ElastiCache	Managed Redis or Memcached in-memory cache.	Offload RDS; session store; Redis for richer data structures + persistence options.
MemoryDB	Redis-compatible, durable, multi-AZ in-memory database.	When you need Redis API + stronger durability than classic cache.
Redshift	Petabyte-scale columnar data warehouse. SQL analytics, Spectrum to S3.	OLAP, not OLTP. Don't put checkout transactions here.
DocumentDB	MongoDB-compatible managed document DB.	When the app already speaks Mongo APIs.
Neptune	Managed graph database (property graph / RDF).	Social graphs, recommendations, fraud rings.
Keyspaces	Managed Apache Cassandra-compatible.	Wide-column, Cassandra workloads.
QLDB	Ledger: immutable, cryptographically verifiable transaction log.	Systems of record, audit history.
OpenSearch Service	Search and log analytics (fork of Elasticsearch).	Full-text search, not a system of record.
RDS Proxy	Connection pooling in front of RDS/Aurora — vital for Lambda + RDS.	Lambda can exhaust DB connections without it.

5.1 RDS vs DynamoDB decision tree

If you need…	Choose	Notes
Joins, transactions, ad-hoc SQL, mature ORM	RDS / Aurora	Vertical scale + read replicas; Aurora scales storage auto
Huge scale, simple access by key, spiky traffic	DynamoDB	Model access patterns first; avoid scans
In-memory microsecond reads	ElastiCache	Cache aside in front of DB
Warehouse / BI across years of events	Redshift (+ S3/Glue)	ETL/ELT from OLTP into warehouse
Multi-region active-active key-value	DynamoDB Global Tables	Conflict = last-writer-wins style; design carefully

DynamoDB interview trap: **hot partitions** (bad partition key). RCU/WCU are per table and split across partitions. On-demand mode absorbs unknown spikes. Use **single-table design** only if you understand access patterns.

6. Networking & content delivery

Service	What it is / primary use	Interview / exam notes
VPC	Your isolated virtual network. CIDR, subnets (public/private), route tables, IGW, NAT.	Draw this in every architecture question. /16 VPC, /24 subnets is a common pattern, not a law.
Subnets	AZ-scoped IP ranges. Public = route to Internet Gateway. Private = no IGW route.	Public subnet ≠ "insecure"; NACLs/SGs still apply. Put DBs in private subnets.
Internet Gateway (IGW)	Horizontally scaled VPC ↔ internet for resources with public IPs.	Needed for public subnets. One IGW per VPC.
NAT Gateway / NAT instance	Private subnet outbound internet (patches, APIs) without inbound from internet.	NAT Gateway is HA per AZ (deploy one per AZ). Costly (hours + GB). NAT instance is DIY.
Security Groups	Stateful L3/L4 firewall on ENIs. Allow rules only. Return traffic auto-allowed.	Default deny inbound. Reference other SGs. Preferred over NACLs for most app control.
Network ACLs	Stateless subnet firewall. Allow and deny. Ephemeral ports matter.	Use for coarse subnet allow/deny (e.g. block a bad IP). Don't duplicate all SG rules.
Route 53	DNS: public/private hosted zones. Routing: simple, weighted, latency, failover, geolocation, multi-value.	Alias records to ALB/CloudFront/S3. Health checks + failover.
CloudFront	CDN. Cache at edge. Origins: S3, ALB, custom. OAC to lock S3 origin.	HTTPS, geo restriction, signed URLs/cookies, Lambda@Edge / CloudFront Functions.
Global Accelerator	Anycast static IPs, AWS backbone to nearest Region — not a cache.	Use when you need static IPs + global anycast, not static file caching (that's CloudFront).
Direct Connect	Private dedicated fiber from on-prem/colo to AWS. Predictable bandwidth.	Vs VPN: DX is private/consistent; VPN is encrypted over internet, faster to set up.
Site-to-Site VPN / Client VPN	IPsec from on-prem; Client VPN for remote users into VPC.	Often backup path if DX fails.
Transit Gateway	Hub-and-spoke to connect many VPCs and on-prem (DX/VPN) without full mesh peering.	Scale after a handful of VPC peerings.
VPC Peering	Connect two VPCs (same or cross-account/region). Non-transitive.	A-B and B-C does not imply A-C. CIDRs cannot overlap.
PrivateLink / VPC endpoints	Interface endpoints (PrivateLink) and Gateway endpoints (S3, DynamoDB) — private AWS API access.	Avoid NAT for S3/DynamoDB with gateway endpoints (cheaper, private).
Elastic IP / IPv6	Static public IPv4. IPv6 dual-stack increasingly common.	Unused Elastic IPs cost money. Prefer ALB + DNS over sprinkling EIPs.

6.1 Three-tier VPC you should be able to sketch

Public subnets (2+ AZs): ALB, optionally NAT Gateways, bastion/SSM endpoints (prefer SSM over bastion).
Private app subnets: EC2/ECS/Lambda ENIs, no public IPs, outbound via NAT if needed.
Private data subnets: RDS, ElastiCache; SG only allows app SG on 5432/3306/6379.
Route 53 → CloudFront (optional) → ALB → app ASG → RDS Multi-AZ. That drawing scores interviews.

7. Application integration & APIs

Service	What it is / primary use	Interview / exam notes
SQS	Managed queue. Standard (at-least-once, best-effort order) vs FIFO (exactly-once processing within limits, order per group).	Visibility timeout, DLQ, long polling. Decouple producers/consumers.
SNS	Pub/sub topics. Fan-out to SQS, Lambda, HTTP, email, SMS, mobile push.	SNS + SQS fan-out is a classic pattern. FIFO topics exist.
EventBridge	Event bus (default AWS events, custom, SaaS). Rules route to many targets. Scheduler too.	Better than polling CloudWatch Events name in 2019 — still the same family. Decouple microservices.
API Gateway	REST, HTTP, and WebSocket APIs. Auth (IAM, Cognito, Lambda authorizer), throttling, API keys, WAF.	HTTP API is cheaper/faster for simple JWT/IAM; REST API has more features (API keys, usage plans, request validation).
AppSync	Managed GraphQL with resolvers to DynamoDB, Lambda, Aurora, OpenSearch, HTTP.	Real-time subscriptions. Mention when the frontend wants GraphQL.
Step Functions	Serverless orchestration (JSON state machines). Standard vs Express workflows.	Replace nested Lambda callbacks; retries, catch, parallel, human approval.
Kinesis Data Streams	Real-time streaming ingest, ordered per shard, multiple consumers.	Vs SQS: streams retain data for replay; multiple apps can read independently.
Amazon MQ	Managed ActiveMQ / RabbitMQ for legacy JMS/AMQP apps.	When you cannot rewrite to SQS/SNS.
EventBridge Pipes / Kafka (MSK)	MSK = managed Kafka. Pipes connect sources to targets with filtering/enrichment.	Kafka shops → MSK; AWS-native new apps often Kinesis/EventBridge/SQS.

7.1 Messaging cheat: SQS vs SNS vs EventBridge vs Kinesis

Service	Pattern	Pick when
SQS	Queue (pull)	Work buffer, competing consumers, smoothing spikes
SNS	Topic (push pub/sub)	Fan-out same message to many subscribers
EventBridge	Event bus + rules	Many event types, SaaS/AWS events, content filtering
Kinesis	Stream	High ingest, replay, analytics, multiple independent readers

8. Monitoring, logging, and audit

Service	What it is / primary use	Interview / exam notes
CloudWatch	Metrics, logs, alarms, dashboards, Logs Insights, Contributor Insights, Synthetics, RUM.	Custom metrics, high-resolution, alarm → SNS → incident. Logs ingestion is a cost trap.
X-Ray / CloudWatch Application Signals	Distributed tracing across Lambda, APIs, ECS.	Latency breakdown, service map. Mention OpenTelemetry as modern direction.
CloudTrail	API audit (management events; data events optional for S3/Lambda).	Who changed the SG? That's CloudTrail, not CloudWatch.
Config	Resource inventory + history + conformance packs.	"Was this bucket public last Tuesday?"
Health Dashboard / Personal Health	AWS service issues vs issues that affect your resources.	Don't confuse with CloudWatch alarms on your app.

Observability triad: **metrics** (is it broken?), **logs** (why?), **traces** (where in the hop?). Add **alarms with runbooks** and **DLQs** so failures are visible.

9. DevOps, IaC, and CI/CD

Service	What it is / primary use	Interview / exam notes
CloudFormation	Native IaC templates (YAML/JSON). Stacks, change sets, drift detection, StackSets.	Still the common "pure AWS IaC" answer.
CDK	Define AWS infra in TypeScript/Python/Java/Go/C#; synthesizes CloudFormation.	Popular in modern teams; say "CDK synthesizes to CFN."
Terraform / OpenTofu (ecosystem)	Multi-cloud IaC widely used with AWS provider.	Interviews accept this; know state, plan/apply, modules.
CodeCommit / CodeBuild / CodeDeploy / CodePipeline	AWS-native Git, build, deploy, pipeline. CodeDeploy also deploys to EC2/ECS/Lambda.	Many companies use GitHub Actions/GitLab instead — say you can map the stages.
Systems Manager (SSM)	Session Manager (no SSH bastion), Patch Manager, Run Command, Parameter Store, Fleet Manager.	SSM Session Manager is the secure-admin answer.
Service Catalog / Control Tower	Approved product portfolios; Control Tower for multi-account landing zone.	Enterprise governance answers.

10. Analytics, ML, migration, and other high-frequency names

You do not need deep internals for all of these. You **do** need a crisp one-liner and when you'd pick them.

Service	What it is / primary use	Interview / exam notes
Athena	Serverless SQL over files in S3 (via Glue Data Catalog / Iceberg etc.).	Ad-hoc log/BI queries without a warehouse cluster.
Glue	Serverless ETL, crawlers, Data Catalog, Spark jobs.	Catalog is the hive metastore analog for S3 data lakes.
EMR	Managed Hadoop/Spark/Hive clusters.	Heavy custom Spark vs Glue's serverless Spark tradeoff.
Kinesis Firehose / Data Firehose	Fully managed stream delivery to S3, Redshift, OpenSearch, Splunk, with optional Lambda transform.	Near-real-time ingest without managing shards like Streams.
QuickSight	BI dashboards (SPICE engine).	Business reporting on Redshift/Athena/RDS.
SageMaker	Build, train, tune, deploy ML models. Notebooks, pipelines, endpoints.	Don't train on a random EC2 unless you must.
Rekognition / Comprehend / Textract / Transcribe / Polly / Translate / Bedrock	Vision, NLP, OCR, speech-to-text, TTS, translation, foundation models via API.	Bedrock = managed FMs; Rekognition = images/video.
DMS	Database Migration Service — homogeneous/heterogeneous DB migrations, CDC.	Oracle→Aurora, on-prem→RDS with minimal downtime.
DataSync / Snow Family / Transfer Family	DataSync: online NFS/SMB/S3 sync. Snowball/Snowmobile: offline petabytes. Transfer: SFTP/FTPS/FTP on S3.	Migration questions: bandwidth vs truck (Snow).
STS	Security Token Service — temporary credentials, AssumeRole, federation.	Every role assumption goes through STS.
Certificate Manager, Secrets Manager	TLS certs; secret rotation.	Covered in security — listed because they appear in almost every app design.
Amplify	Frontend hosting + auth/API/storage helpers for web/mobile.	Startups/hackathons; not a substitute for a VPC design.

11. Well-Architected Framework & shared responsibility

11.1 Shared responsibility (say it in 20 seconds)

AWS is responsible for security *of* the cloud: hardware, software, networking, and facilities that run AWS services. **You** are responsible for security *in* the cloud: IAM, guest OS (EC2), network config, encryption, application code, and classifying data. For **managed/serverless** (Lambda, RDS, S3), AWS patches more of the stack; you still own access policies, encryption settings, and your code/queries.

11.2 Six pillars

Pillar	Goal	Concrete AWS examples
Operational excellence	Run, monitor, evolve	IaC, CI/CD, CloudWatch, runbooks, game days
Security	Confidentiality, integrity, identity	IAM least privilege, KMS, SG, CloudTrail, WAF, GuardDuty
Reliability	Recover from failure, scale	Multi-AZ, Auto Scaling, SQS buffering, backups, Route 53 failover
Performance efficiency	Right resource, evolve with tech	Right-size instance, Aurora, CloudFront, caching, Graviton
Cost optimization	Avoid waste	Savings Plans, Spot, S3 classes, NAT reduction, Cost Explorer, Budgets
Sustainability	Reduce environmental impact	Higher utilization, managed services, efficient Regions, Graviton

11.3 High availability vs disaster recovery

- **HA:** survive AZ failure in a Region (Multi-AZ RDS, ASG across AZs, NAT per AZ).
- **DR:** survive Region failure. Strategies: backup/restore, pilot light, warm standby, multi-site active/active.
- **RTO:** how long until service is back. **RPO:** how much data you can lose.
- S3 CRR, Aurora Global, DynamoDB Global Tables, Route 53 failover are common DR building blocks.

12. Service comparison cheat sheets

12.1 Storage: S3 vs EBS vs EFS vs instance store

	S3	EBS	EFS
Type	Object	Block	File (NFS)
Access	HTTP API / SDKs	One instance (typical)	Many instances
Scope	Regional (multi-AZ implicit)	Single AZ volume	Regional multi-AZ
Use	Assets, data lake, backups	OS disk, databases on EC2	Shared content, home dirs, CMS
Not for	OS boot (use EBS)	Unbounded objects / web assets	Windows SMB (use FSx)

12.2 Load balancers

	ALB	NLB	GWLB
Layer	7 (HTTP/HTTPS, gRPC)	4 (TCP/UDP/TLS)	3 (transparent bump-in-wire)
Features	Path/host, Cognito, WAF, sticky	Static IP, extreme pps, PrivateLink	Third-party firewalls/IDS
Target	Web/microservices	Gaming, IoT, non-HTTP	Security appliances

12.3 Encryption at rest — default talking points

- S3: encrypt by default (SSE-S3); use SSE-KMS when you need key policy/audit.
- EBS: encrypt volumes + snapshots; copy snapshot to share encrypted data across accounts.
- RDS: enable encryption at creation (or snapshot-copy to encrypt later).
- In transit: TLS everywhere (ALB, CloudFront, ACM). VPC-only traffic for data tiers.

13. Common architectures you should be able to draw

13.1 Highly available web application

Users → Route 53 → (optional CloudFront) → ALB in public subnets (2 AZs) → EC2/ECS in private ASG → RDS Multi-AZ (or Aurora) in data subnets + ElastiCache. NAT Gateway per AZ for outbound. S3 for static assets (origin via CloudFront + OAC). IAM roles on compute. Secrets in Secrets Manager. Logs to CloudWatch; API audit via CloudTrail; alarms to SNS.

13.2 Serverless API

Client → CloudFront → API Gateway (Cognito or JWT authorizer) → Lambda → DynamoDB. Async side effects: Lambda → SQS → another Lambda. Files: presigned S3 URLs. EventBridge for SaaS/AWS events. Step Functions if the workflow has many steps/retries. Observability: structured logs, X-Ray, DLQs.

13.3 Decoupled processing / fan-out

Producer → SNS topic → multiple SQS queues (email, analytics, fulfillment) → workers (ECS/Lambda). Each queue has a DLQ. Idempotent consumers. This is the standard “don’t couple microservices with sync HTTP only” answer.

13.4 Data lake / analytics

Ingest (Kinesis Firehose / DMS / DataSync) → S3 (bronze/silver/gold prefixes) → Glue Catalog + ETL → Athena ad-hoc and/or Redshift for BI → QuickSight. Lake Formation for permissions in mature setups.

13.5 Hybrid

On-prem DC → Direct Connect (VPN backup) → Transit Gateway → multiple VPCs (prod/dev/shared services). Route 53 private zones. Storage Gateway or DataSync for backups to S3. IAM Identity Center for humans.

14. Interview questions and model answers

Speak in requirements: availability, consistency, latency, cost, ops, security. Name 1–2 services, one tradeoff, one failure mode.

What happens if an AZ fails in your web architecture?

ALB and ASG are in at least two AZs, so the balancer stops sending traffic to unhealthy targets and the ASG launches capacity in the surviving AZs. RDS Multi-AZ fails over to the standby (brief downtime). NAT must exist in each AZ or private egress in the failed AZ dies. Single-AZ NAT, single-AZ RDS, or IP hardcoding are typical bugs.

S3 vs EBS vs EFS — which do I pick?

S3 for objects and data lakes (no OS filesystem semantics). EBS as the disk for an EC2 instance or self-managed DB. EFS when many Linux compute units need a shared POSIX filesystem. Windows SMB → FSx for Windows File Server.

How does IAM authentication differ from authorization?

Authentication proves who you are (user, role session, federated identity). Authorization is what you're allowed to do (policies). STS issues temporary creds after you assume a role; then every API call is authorized against policies, boundaries, and SCPs.

Why not put a database in a public subnet?

It expands the attack surface (security groups can be mis-opened, public IP reachable). Put RDS in private subnets; only the app SG can connect. Admins use SSM or a tightly controlled bastion, not 0.0.0.0/0 on 3306.

Lambda connected to RDS is timing out or exhausting connections. Why?

High concurrency creates many DB connections. Use RDS Proxy, connection pooling, or move to DynamoDB if access patterns fit. Also: Lambda in VPC needs subnet/NAT or VPC endpoints; misconfigured ENI/SG causes timeouts.

Standard SQS vs FIFO?

Standard: nearly unlimited throughput, at-least-once, best-effort ordering — design idempotent consumers. FIFO: per message-group ordering, exactly-once processing within a 5-minute dedup window, limited throughput unless you scale with group IDs. Payments/ledger-like order → FIFO; clickstream → standard.

How do you encrypt data on AWS?

In transit: TLS (ACM on ALB/CloudFront). At rest: S3 SSE, EBS encryption, RDS encryption, DynamoDB encryption. Customer-managed KMS keys when you need rotation, key policies, and CloudTrail on key use. Never put secrets in AMIs or Git; use Secrets Manager/SSM.

What is a security group vs NACL?

SG: stateful, ENI-level, allow-only, return traffic automatic — primary app firewall. NACL: stateless, subnet-level, allow and deny, must allow ephemeral ports — extra subnet boundary or explicit denies.

How would you design for 10x traffic on Black Friday?

Stateless app tier + Auto Scaling on ALB request count/CPU. Cache (CloudFront, ElastiCache). Queue bursts (SQS). DynamoDB on-demand or RDS read replicas / Aurora. Load-test. Pre-warm known bottlenecks (NLB/ALB usually autoscale; NAT and RDS don't magically infinitely scale).

EC2 instance was compromised. What do you do?

Isolate (SG deny, stop/detach). Snapshot volumes for forensics. Rotate IAM creds/roles that were on the instance, rotate secrets. CloudTrail + VPC Flow Logs + GuardDuty for blast radius. Rebuild from known-good AMI (cattle, not pets). Fix root cause (IMDSv2, SSM, least privilege, patching).

CloudWatch vs CloudTrail vs Config?

CloudWatch: operational metrics/logs/alarms of running systems. CloudTrail: API history (who called what). Config: how resources were configured over time and whether they comply with rules.

When do you choose Kubernetes (EKS) over ECS?

EKS when you need k8s APIs, portability, or existing Helm/operators. ECS/Fargate when you want less cluster overhead on AWS-only containers. Don't pick EKS just because it's trendy — mention ops cost of the control/data plane.

What is eventual consistency in DynamoDB?

After a write, a strongly consistent read (same region) can be requested at extra cost/latency. Default reads may briefly see stale data. Global Tables are multi-region replicating; conflicts need an application strategy. Don't treat DynamoDB like a SQL JOIN store.

How do you keep S3 private but let users upload/download?

Block Public Access on. App generates presigned URLs (or CloudFront signed URLs/cookies) using a role that can Get/Put that prefix. Never make the bucket public for "convenience." Use OAC if CloudFront is the only reader.

Spot instances — safe for production?

Yes for fault-tolerant, interruptible work (containers with draining, batch, CI, stateless workers with queues). No for single-node stateful DBs without replication. Use mixed-instances ASG, interruption notices, and checkpointing.

How do accounts and Organizations help security?

Blast-radius isolation (prod/dev/sandbox/logs/security). SCPs prevent even root-in-member from disabling GuardDuty/leaving org (with care). Central Log Archive account. Identity Center maps workforce to roles per account.

What is the difference between horizontal and vertical scaling?

Vertical: bigger instance/db class (simple, ceiling, bigger blast radius). Horizontal: more instances behind a balancer (needs statelessness or shared state in RDS/Dynamo/Redis). ASG is horizontal; changing RDS instance size is vertical.

NAT Gateway costs are high. What do you do?

VPC gateway endpoints for S3 and DynamoDB. Interface endpoints for other AWS APIs. Reduce cross-AZ traffic. Don't send CloudFront/S3 downloads through NAT. Consider NAT per AZ only where needed; watch GB processed.

Explain Multi-AZ RDS vs read replicas.

Multi-AZ: synchronous standby for HA/failover, not for extra read throughput (you don't read the standby in classic RDS Multi-AZ). Read replicas: async, scale reads, can be promoted; replica lag is the tradeoff. Aurora replicas share storage and are a common read-scale answer.

What would you put in a 30-second "AWS strengths" pitch?

Breadth of reliable primitives (compute, data, global network), strong IAM and VPC isolation model, managed services that shift undifferentiated ops, and a mature ecosystem. Weakness to acknowledge: complexity and cost surprises (data transfer, NAT, logs) if ungoverned.

15. Quick study checklist

Can you explain each of these without notes?

- ■ Region vs AZ vs edge location vs account
- ■ IAM user vs role vs policy vs SCP vs permission boundary
- ■ Security group vs NACL; public vs private subnet; IGW vs NAT
- ■ ALB vs NLB; ASG + launch template
- ■ S3 classes, versioning, encryption, Block Public Access, presigned URLs
- ■ EBS vs EFS vs S3 vs FSx
- ■ RDS Multi-AZ vs read replica vs Aurora vs DynamoDB vs ElastiCache
- ■ Lambda limits; API Gateway types; SQS vs SNS vs EventBridge vs Kinesis
- ■ CloudWatch vs CloudTrail vs Config vs GuardDuty
- ■ CloudFront vs Global Accelerator
- ■ KMS envelope encryption; Secrets Manager vs Parameter Store
- ■ RTO/RPO and four DR patterns
- ■ Well-Architected six pillars + shared responsibility
- ■ Draw: 3-tier HA web app and a serverless API

Most-used services — one-line recap

Cluster	Remember as
IAM / Organizations / KMS	Who can do what; multi-account; keys
VPC + SG + ALB + ASG + EC2	Network + HA compute
S3 + CloudFront + Route 53	Objects, CDN, DNS
RDS/Aurora + DynamoDB + ElastiCache	Relational, key-value, cache
Lambda + API Gateway + SQS/SNS/EventBridge	Serverless compute + APIs + decoupling
ECS/Fargate or EKS + ECR	Containers
CloudWatch + CloudTrail + Config	Operate, audit, comply
CloudFormation/CDK + SSM	IaC and safe operations

Honest scope note

AWS has hundreds of services. Interviews for general backend/cloud/DevOps roles cluster around the services in this PDF. If the job is data engineering, go deeper on Glue, Spark/EMR, Redshift, Lake Formation, MSK. If it is security, go deeper on Organizations, SCPs, GuardDuty, Security Hub, Detective, Network Firewall, IAM Access Analyzer. Always pair a service name with a requirement and a failure mode.

This study aid is independent and unofficial. Service names, limits, and pricing change — verify against current AWS documentation before production designs or certification exams.